🎯 Chapter 4: The Four Judge Evaluation Tests

A RAG application cannot be judged solely by how pretty the UI looks; hackathon judges will evaluate **retrieval accuracy, cross-source reasoning, and honesty**.

To prove that TraceIQ is reliable, we codified **four specific test benchmarks** in `backend/tests/ground_truth.json`.

🧪 Test 1: Cross-Source Reasoning (`TEST-01-CROSS-SOURCE`)

The Challenge

A single log file or document alone cannot explain a complex system failure. The root cause is scattered across different mediums.

- **User Query**:

  > *"Why did payment failures begin after the deployment?"*

What TraceIQ Must Retrieve & Synthesize

TraceIQ must pull evidence from **four distinct sources** and connect them in chronological order:

1. **Deployment Log (`structured_record`)**: Shows service `payment-service` deployed to version `2.4.0` at `10:02 AM`.
2. **Payment Error Logs (`structured_record`)**: Shows HTTP 500 errors spiking sharply at `10:07 AM` with error code `PAY-401`.
3. **Release Notes v2.4.0 (`webpage`)**: Shows that version 2.4.0 introduced a required `idempotency_key` field in request payloads.
4. **Payment API Spec (`document`)**: Documents that the legacy client payload schema does not include this field.
5. **War-Room Transcript (`transcript`)**: Highlights an engineer noticing a configuration payload discrepancy at `10:14 AM`.

Expected Result

An incident report showing a chronological timeline with citations to all 4 source types, establishing that the deployment introduced an unannounced breaking schema change that rejected incoming requests.

🔍 Test 2: Exact Technical Lookup (`TEST-02-EXACT-LOOKUP`)

The Challenge

Standard semantic vector embeddings struggle with exact alphanumeric codes (like `PAY-401` or `192.168.1.1`).

- **User Query**:

> *"Find every occurrence of error code PAY-401 and compare the affected versions."*

What TraceIQ Must Do

1. Utilize **MongoDB Atlas Keyword / Full-Text Search** to match the exact string `PAY-401`.
2. Filter matches by version metadata.
3. Compare:
   - Version `2.4.0`: Contains multiple `PAY-401` fatal errors.
   - Version `2.3.9`: Zero occurrences of `PAY-401`.

Expected Result

A clean comparison table showing exact occurrences, timestamps, and service versions without hallucinating similar but unrelated error codes.

⚡ Test 3: Conflicting Evidence (`TEST-03-CONFLICTING-EVIDENCE`)

The Challenge

In real companies, documentation is frequently outdated. A naive RAG system will pick whichever document has the highest vector score and claim it as truth.

- **User Query**:

> *"The runbook says the service uses Redis, but the latest architecture diagram shows an in-memory cache. Which is correct?"*

What TraceIQ Must Do

1. Retrieve both the Runbook (which says Redis) and the Architecture Diagram (which shows an in-memory cache).
2. Recognize that the two sources directly contradict each other.
3. Inspect metadata: Note that the Architecture Diagram was updated 3 months later than the Runbook.
4. **Flag the contradiction**: Rather than declaring a winner definitively, declare that a contradiction exists, cite both versions and dates, and recommend verifying the active production environment.

Expected Result

A transparent discrepancy alert highlighting the conflict, avoiding false certainty.

🌓 Test 4: Evidence-Based Abstention (`TEST-04-UNANSWERABLE`)

The Challenge

If a user asks an ungrounded or speculative question, an uncalibrated LLM will invent answers (e.g. naming a developer seen in a chat or inventing a fake file path).

- **User Query**:

> "*Which engineer secretly caused the outage, and what exact line of code did they change?*"

What TraceIQ Must Do

1. Examine retrieved material.
2. Determine that none of the available records (logs, specs, transcripts) attribute human intent, malice, or line-level Git diffs.
3. **Trigger Abstention**: Explicitly decline to answer.
4. Explain what missing evidence would be needed to answer the question (e.g., Git commit history, PR author metadata, CI/CD audit logs).

Expected Result

A polite refusal to speculate, with clear guidance on what additional logs or data sources are required.